

Sequence-to-Sequence Models: Intro to Attention

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

June 18, 2026

1. Motivation
2. Learning Outcomes
3. Seq2Seq Architecture
4. Encoder-Decoder Framework
5. Applications of Seq2Seq Models
6. Challenges and Limitations
7. Attention Mechanisms: Motivation and Introduction
8. How Attention Works
9. Benefits of Attention in NLP
10. Summary and Future Directions

Why Do We Need Seq2Seq and Attention?

- ▶ Many real-world problems require transforming one sequence to another:
 - Translation: "Bonjour" $\rightarrow$ "Hello"
 - Dialogue systems: Question $\rightarrow$ Response
 - Speech: Audio $\rightarrow$ Text

Standard RNNs struggle with input/output sequences of different lengths and long-term dependencies.

Seq2Seq models + attention solve this with a powerful encoder-decoder framework.

By the end of this session, you should be able to:

- ▶ Explain the Seq2Seq architecture and encoder-decoder framework
- ▶ Understand the bottleneck problem in fixed-size representations
- ▶ Describe the motivation for and core idea behind attention mechanisms
- ▶ Appreciate how attention improves performance in NLP tasks
- ▶ Recognize future directions in attention-based modeling

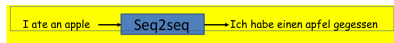
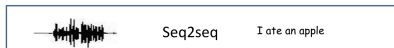
Seq2Seq Architecture

Key Idea: Map input sequence $\rightarrow$ intermediate vector $\rightarrow$ output sequence.

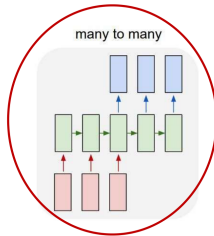
- ▶ **Encoder RNN:** Processes input sequence and compresses it into a **fixed-length vector** (context).
- ▶ **Decoder RNN:** Generates output sequence from the context vector.

Applications:

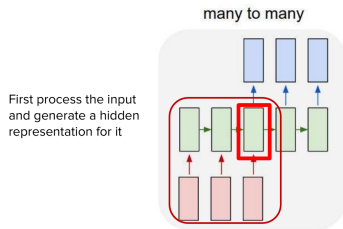
- ▶ Machine translation
- ▶ Summarization
- ▶ Dialogue systems
- ▶ Speech recognition



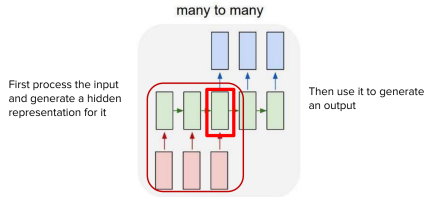
- Sequence goes in, sequence comes out
- No notion of “time synchrony” between input and output
 - May even not even maintain order of symbols
 - E.g. “I ate an apple” ↔ “Ich habe einen apfel gegessen”
 - Or even seem related to the input
 - E.g. “My screen is blank” ↔ “Please check if your computer is plugged in.”



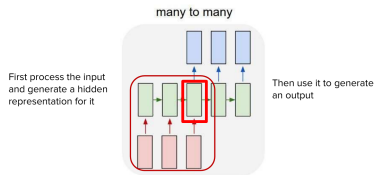
Delayed sequence to sequence



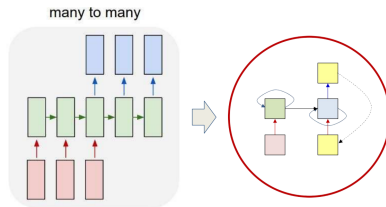
Delayed sequence to sequence



Delayed sequence to sequence

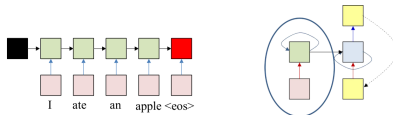


Problem: Each word that is output depends only on current hidden state, and not on previous outputs

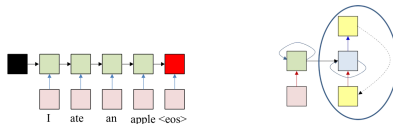


- *Delayed* Sequence to sequence
 - Delayed *self-referencing* sequence-to-sequence

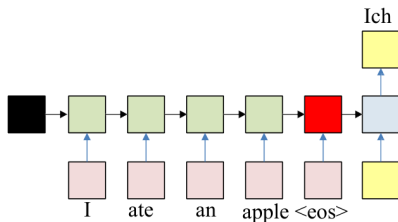
The “simple” translation model



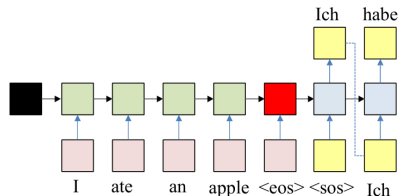
- ▶ The input sequence feeds into a recurrent structure
- ▶ The input sequence is terminated by an explicit <eos> symbol
 - The hidden activation at the <eos> “stores” all information about the sentence



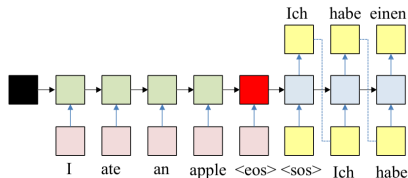
- ▶ The input sequence feeds into a recurrent structure
- ▶ The input sequence is terminated by an explicit <eos> symbol
 - The hidden activation at the <eos> “stores” all information about the sentence
- ▶ Subsequently a second RNN uses the hidden activation as initial state, and <sos> as initial symbol, to produce a sequence of outputs
 - The output at each time becomes the input at the next time
 - Output production continues until an <eos> is produced



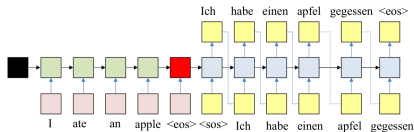
- ▶ The input sequence feeds into a recurrent structure
- ▶ The input sequence is terminated by an explicit <eos> symbol
 - The hidden activation at the <eos> “stores” all information about the sentence
- ▶ Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
 - The output at each time becomes the input at the next time
 - Output production continues until an <eos> is produced



- ▶ The input sequence feeds into a recurrent structure
- ▶ The input sequence is terminated by an explicit <eos> symbol
 - The hidden activation at the <eos> “stores” all information about the sentence
- ▶ Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
 - The output at each time becomes the input at the next time
 - Output production continues until an <eos> is produced

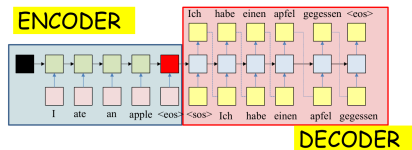


- ▶ The input sequence feeds into a recurrent structure
- ▶ The input sequence is terminated by an explicit <eos> symbol
 - The hidden activation at the <eos> “stores” all information about the sentence
- ▶ Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
 - The output at each time becomes the input at the next time
 - Output production continues until an <eos> is produced



- ▶ The input sequence feeds into a recurrent structure
- ▶ The input sequence is terminated by an explicit <eos> symbol
 - The hidden activation at the <eos> “stores” all information about the sentence
- ▶ Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs
 - The output at each time becomes the input at the next time
 - Output production continues until an <eos> is produced

The “simple” translation model



- ▶ The recurrent structure that extracts the hidden representation from the input sequence is the **encoder**
- ▶ The recurrent structure that utilizes this representation to produce the output sequence is the **decoder**

The Bottleneck Problem

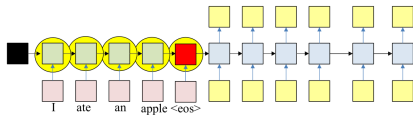
Fixed-length context vector = information bottleneck

- ▶ Encoder must **compress entire input sequence** into a single vector
- ▶ Longer or more complex inputs → information loss
- ▶ Decoder relies solely on that vector to produce outputs

Leads to poor performance on long sentences or tasks requiring high context awareness

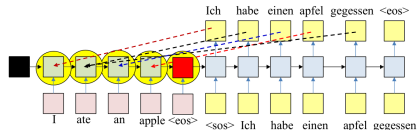
- ▶ All the information about the input sequence is embedded into a single vector.
 - The “hidden” node layer at the end of the input sequence.
 - This one node is “overloaded” with information, particularly if the input is long.

A problem with this framework



- In reality: All hidden values carry information.
 - Some of which may be diluted by the time we get to the final state of the encoder.

A problem with this framework

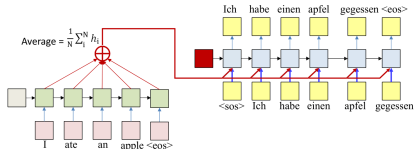


- ▶ Every output is related to the input directly.
- ▶ Not sufficient to have the encoder hidden state to only the initial state of the decoder.
- ▶ Misses the direct relation of the outputs to the inputs.

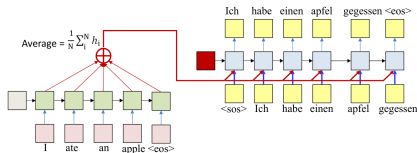
Realization: We Need More Context!

Decoder should have access to all encoder states, not just the final one.

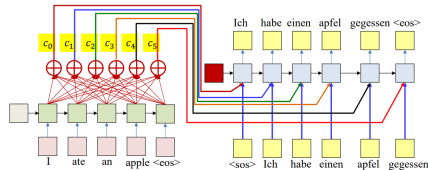
- ▶ This inspired the development of the **attention mechanism**.
- ▶ Instead of passing only the final state, allow the decoder to “*look back*” at **all input positions**.



- ▶ Simple solution: Compute the average of all encoder hidden states.
- ▶ Input this average to every stage of the decoder.
- ▶ The initial decoder hidden state is now separate from the encoder and may be a learnable parameter.



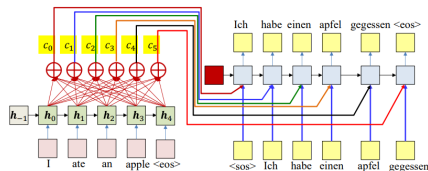
- ▶ Problem: The average applies the same weight to every input.
- ▶ It supplies the same average to every output word.
- ▶ In practice, different outputs may be related to different inputs.
 - E.g., "Ich" is most related to "I", and "habe" and "gegessen" are both most related to "ate".



Solution: Use a *different* weighted average for each output word

- The weighted average provided for the k th output word is:

$$c_t = \frac{1}{N} \sum_i w_i(t) h_i$$



$$c_t = \frac{1}{N} \sum_{i=1}^N w_i(t) h_i$$

- ▶ This solution will work if the weights w_{ki} can somehow be made to “focus” on the right input word
 - E.g., when predicting the word “apfel”, $w_3(4)$, the weight for “apple” must be high while the rest must be low
- ▶ How do we generate such weights??

Introduction to Attention Mechanism

Core Idea: Let the decoder **focus on different parts of the input** sequence at each step of decoding.

- ▶ At each decoding step, compute a **weighted sum** over all encoder hidden states.
- ▶ Weights reflect **relevance** of each input word to the current output word.

“Soft search” over inputs → more context-awareness.

1. Alignment Score:

$$e_{t,s} = \text{score}(h_t^{\text{dec}}, h_s^{\text{enc}})$$

2. Attention Weights (Softmax):

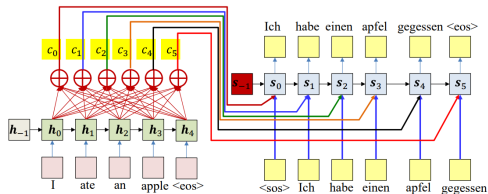
$$\alpha_{t,s} = \frac{\exp(e_{t,s})}{\sum_{s'} \exp(e_{t,s'})}$$

3. Context Vector:

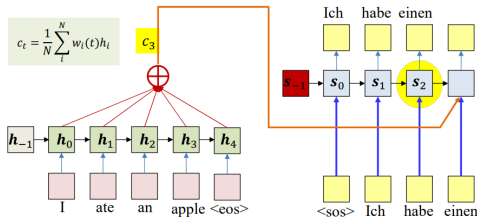
$$c_t = \sum_s \alpha_{t,s} h_s^{\text{enc}}$$

4. Decoder Input:

$$y_t = \text{Decoder}(y_{t-1}, h_{t-1}^{\text{dec}}, c_t)$$

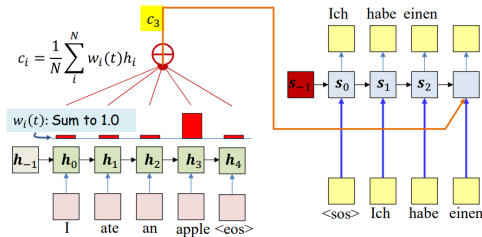


- **Attention weights:** The weights are dynamically computed as functions of decoder state.
- **Expectation:** if the model is well-trained, this will automatically “highlight” the correct input.
- But how are these computed?

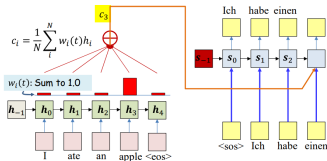


- ▶ The “attention” weights $w_i(t)$ must be computed from available information at time t .
- ▶ The primary information is s_{t-1} (the state at time $t - 1$).
- ▶ Also, the input word at time t , but generally not used for simplicity.

$$c_t = \sum_{i=1}^N w_i(t) h_i$$



- ▶ The weights must be positive and sum to 1.0 (i.e., be a distribution).
- ▶ Ideally, they must be high for the most relevant inputs for the i -th output and low elsewhere.
- ▶ **Solution: A two step weight computation**
 - First compute raw weights (which could be +ve or -ve).
 - Then softmax them to convert them to a distribution.



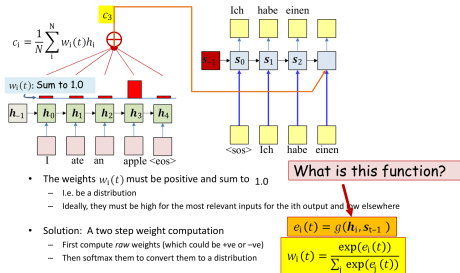
- The weights $w_i(t)$ must be positive and sum to 1.0
 - i.e. be a distribution
 - ideally, they must be high for the most relevant inputs for the i th output and low elsewhere
- Solution: A two step weight computation
 - First compute raw weights (which could be +ve or -ve)
 - Then softmax them to convert them to a distribution

$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

1. The attention framework computes a different “context” vector at each output step?
2. The context vector is chosen as the hidden representation of the word with the highest weight?
3. The weight is a function of the encoder representation and the most recent decoder state?

1. The attention framework computes a different “context” vector at each output step? **True**
2. The context vector is chosen as the hidden representation of the word with the highest weight? **False**
3. The weight is a function of the encoder representation and the most recent decoder state? **True**

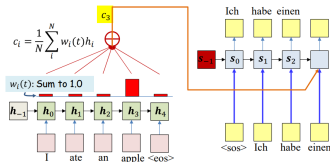


- The weights $w_i(t)$ must be positive and sum to 1.0
 - i.e. be a distribution
 - Ideally, they must be high for the most relevant inputs for the i th output and low elsewhere
- Solution: A two step weight computation
 - First compute *raw* weights (which could be +ve or -ve)
 - Then softmax them to convert them to a distribution

What is this function?

$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$



- Typical options for $g()$ (variables in red must be learned)

$$g(h_i, s_{t-1}) = h_i^T s_{t-1}$$

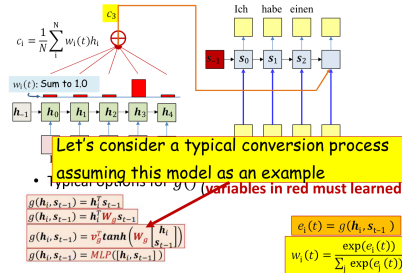
$$g(h_i, s_{t-1}) = h_i^T w_g s_{t-1}$$

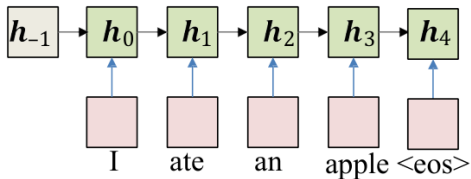
$$g(h_i, s_{t-1}) = v_g^T \tanh \left(w_g \begin{bmatrix} h_i \\ s_{t-1} \end{bmatrix} \right)$$

$$g(h_i, s_{t-1}) = MLP([h_i, s_{t-1}])$$

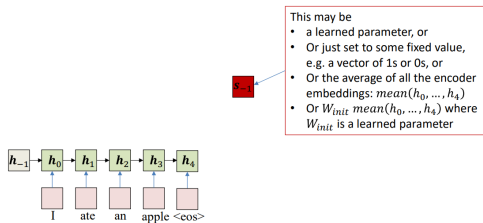
$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$



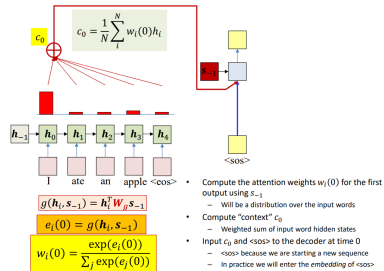


Pass the input through the encoder to produce hidden representations h_i

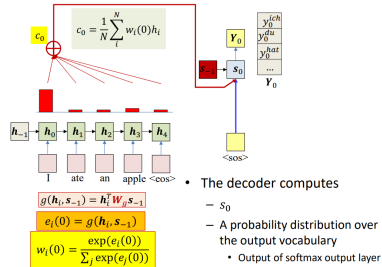


Pass the input through the encoder to produce hidden representations h_i

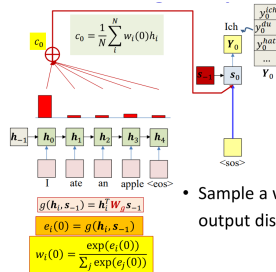
Converting an input: Inference



Converting an input: Inference

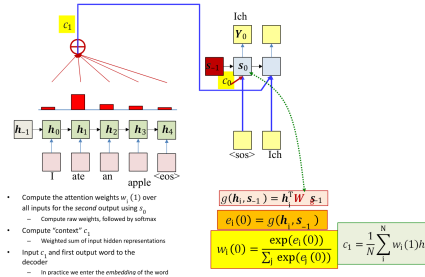


Converting an input: Inference

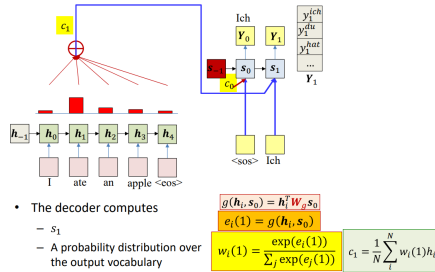


- Sample a word from the output distribution

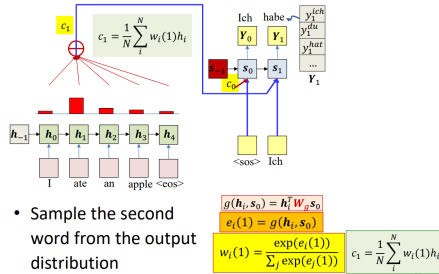
Converting an input: Inference



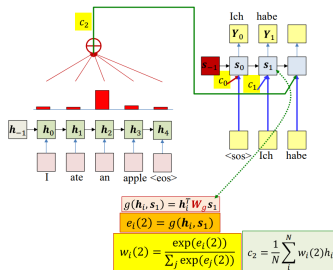
Converting an input: Inference



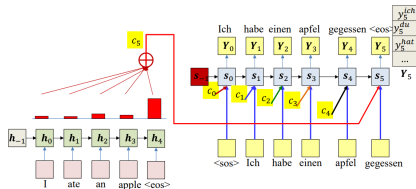
Converting an input: Inference



Converting an input: Inference



Converting an input: Inference

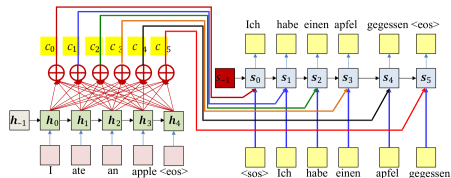


Continue this process until
<eos> is drawn

$$e_i(5) = g(h_i, s_4)$$

$$w_i(5) = \frac{\exp(e_i(5))}{\sum_j \exp(e_j(5))}$$

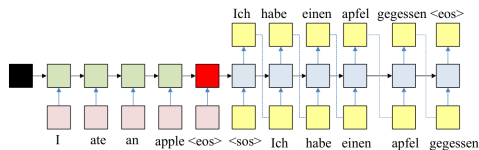
$$c_5 = \frac{1}{N} \sum_i^N w_i(5) h_i$$



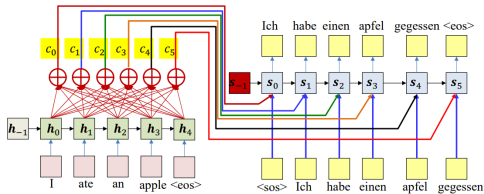
$$e_i(t) = g(h_i, s_{t-1})$$

$$w_i(t) = \frac{\exp(e_i(t))}{\sum_j \exp(e_j(t))}$$

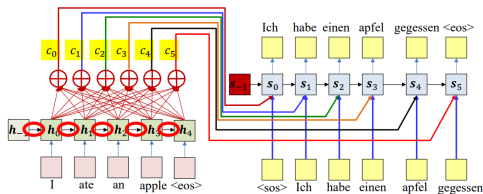
$$c_t = \frac{1}{N} \sum_i w_i(t) h_i$$



- ▶ The input sequence feeds into a recurrent structure
- ▶ The input sequence is terminated by an explicit <eos> symbol
 - The hidden activation at the <eos> "stores" all information about the sentence
- ▶ Subsequently a second RNN uses the hidden activation as initial state to produce a sequence of outputs



- ▶ Encoder recurrently produces hidden representations of input word sequence
- ▶ Decoder recurrently generates output word sequence
 - For each output word the decoder uses a weighted average of the hidden input representations as input "context", along with the recurrent hidden state and the previous output word



- **Problem:** Because of the recurrence, the hidden representation for any word is also influenced by *all* preceding words
 - The decoder is actually paying attention to the sequence, and not just the word

- ▶ Improves translation accuracy
- ▶ Helps handle **longer sequences**
- ▶ Interpretable: shows what the model is focusing on
- ▶ Forms the foundation of **Transformer models**

Led to development of:

- ▶ Bahdanau Attention (Additive, 2014)
- ▶ Luong Attention (Multiplicative, 2015)
- ▶ Self-Attention and Transformers (2017+)

Aspect	Basic Seq2Seq	With Attention
Memory	Single vector (fixed)	Multiple encoder states (dynamic)
Long Sequences	Poor performance	Good performance
Interpretability	Low	High (via attention weights)
Use Cases	Short/medium sequences	Longer, complex tasks

- ▶ Machine Translation
- ▶ Text Summarization
- ▶ Chatbots and Dialog Systems
- ▶ Speech Recognition
- ▶ Question Answering
- ▶ Video Captioning
- ▶ DNA Sequence Modeling

- ▶ Still sequential — **not fully parallelizable**
- ▶ Attention adds computational cost
- ▶ Hard to interpret in large-scale models
- ▶ Might struggle with very long-range dependencies in huge contexts

- ▶ **Transformers:** Fully attention-based, no recurrence
- ▶ **Efficient Attention Models:** Longformer, Reformer, Linformer
- ▶ **Multimodal Attention:** Vision + Text
- ▶ **Memory-Augmented Models**
- ▶ **Structured Attention:** Parses, syntax, and alignment bias

Attention paved the way for GPT, BERT, and LLMs

- ▶ Seq2Seq enables mapping input to output sequences of variable lengths
- ▶ **Bottleneck:** Fixed-size context vector limits learning capacity
- ▶ **Attention:** Improves performance by giving **adaptive access** to input states
- ▶ Attention → Transformer → LLMs
- ▶ Still evolving: From additive attention to self-attention and beyond

These slides have been adapted from:

- ▶ Younes Mourri & Lukasz Kaiser, [Natural Language Processing Specialization, DeepLearning.AI](#)
- ▶ Bhiksha Raj & Rita Singh, [11-785 Introduction to Deep Learning, CMU](#)

Foundational Papers:

- ▶ Sutskever et al. (2014). *Sequence to Sequence Learning with Neural Networks*. NeurIPS.
- ▶ Bahdanau et al. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*.
- ▶ Luong et al. (2015). *Effective Approaches to Attention-based Neural Machine Translation*.
- ▶ Vaswani et al. (2017). *Attention Is All You Need* (Transformer).
- ▶ Chan et al. (2016). *Listen, Attend and Spell* (Speech recognition).

Courses & Tutorials:

- ▶ Stanford CS224n: Lecture 9 (Attention)
- ▶ DeepLearning.ai NLP Specialization (Coursera)
- ▶ Harvard NLP Annotated Transformer:
<http://nlp.seas.harvard.edu/2018/04/03/attention.html>
- ▶ Jay Alammar's blog: "The Illustrated Transformer"